Database (SOC-3020), Fall 2024

Dr. Alessandro Agostini

Homework 2

Issued Sunday 20 October. Due as **zipped archive (.tex, .pdf, images) to eClass by Monday 11 November 2024 at noon (12:00, system's clock)**. Hardcopy: none (upon request for specific cases).

Please note the following:

0. Deadline is *strict*. **Electronic deadline passed without submission implies mark 0 (zero)**. (Equivalently, you will not be admitted to enter the classroom for Quiz 2.)

1. The suggested number of students is 1 to 3. We allow teams formed by maximum 4 members.

1.1 Teams formed by more than 4 members **are not permitted** (students not admitted to Quiz 2).

2. Although teams can be formed by students attending any of the sections 001 to 004, it is preferable all students are in the same section. In any case, students are required to write their **section number** — example: 001 — in the cover of submitted work (see 3 below). NB: Do *not* use the group. Use section!

IMPORTANT: Only students listed in the cover of submitted work are admitted to Quiz 2. In other words, students whose ID, full name and section number do not appear on the cover **are not admitted to enter the classroom for Quiz 2 and are assigned score 0/100 for the homework**.

IMPORTANT: Students in teams who got cheating in Homework #1 can NOT submit this homework.

3. LaTeX's template and Submission. The homework must be set by using LaTeX's template provided (see (*) below). You are asked to submit a **zipped archive** containing the source LaTeX's file (.tex), its compilation (.pdf) using command *pdflatex*, and a folder with all the images included in the .tex file.

(*) The main document (pdf file) you submit must be produced from the source file provided (cf. LaTeX's template in eClass, Week 7) by using LaTeX, specifically one of following versions: 1. https://ctan.org/pkg/texlive (general) or 2. https://ctan.org/pkg/mactex (MacOS users).[a]

[a] If you have serious issues in installing any of such software packages, you are allowed to use OVERLEAF (https://www.overleaf.com), provided that the source file .tex produced is compatibile with TexLive.

4. Only one member of the team submits the document.

IMPORTANT: Multiple submission by members of the same team **are penalized** as follows: For each team member who submits the document already submitted, scoring (of related Quiz 2) is reduced by 5/100 points for all team members. **Example:** A team of three, three members submit: score of Quiz 2 reduced by 10 points (over 100 points) for all team members.

4.1 The **name of the submitted zipped archive as well the name of each submitted file, namely, .tex file and .pdf file** is required to be in the following format:

00X-ID (example: 001-U1910123), where:

- 00X is the section of the team's member, say S, who submits the homework to eCLASS;
- ID is the ID number of S.

5. IMPORTANT: Homework that containts evident elements of **plagiarism or cheating**, either among teams or from existing material (e.g., from the Internet, textbook's Instructor Manual if available online, etc.) implies a mark 0/100 **for all teammates and all future homework assignments.** (Equivalently, you will not be allowed to enter any of in-class quizzes.)

5.1 IMPORTANT: Sharing of homework's file(s) among teams is never permitted at whatever stage of work in progress and it results in a main source of assigned zeros. **You can share ideas, not files!**

PARTICULAR SETTING [same of Homework #1]:

To proceed, you need to install MYSQL and the graphical interface/console *phpMyAdmin* for administering and browsing MySQL (or any compatibly other) database server.

> **IMPORTANT: You are not allowed to use any other *interface* than** *phpMyAdmin*. Please read the slides, ask to a classmate, or contact the Instructor or TA if you need help at this stage.

By using *phpMyAdmin*, create a database (give the name you like; below we used "**UniversityDB**") by importing the 'DDL for MySQL' and 'smallRelationsInsertFile.sql' files that you previously downloaded from textbook's website: https://www.db-book.com/db6/; cf. slides for Week 2 (see Appendix) for installation guidelines. The logical schema of such database is showed in Figure 1 (cf. last page).

PROBLEMS AND EXERCISES:

1. For **each SQL query**, say Q, in the slides discussed in class, exactly from slide 13 in slide-set of Week 5 to slide 43 in slide-set of Week 6 (both slide-sets are available in eClass), do:

   (a) Rewrite Q into your document by using **SQL standard** as written in the slides. Next to the query write the slide-set (example: W5 for slides of Week 5; W6 for slides of Week 6,...) and the slide's page number (you find it bottom-right corner of each slide) of the slide where Q is written. Example: if Q is the SQL query written in the slide with page number 13 of Week 5, you could write something like W5-p13 next to Q.

   (b) By using *phpMyAdmin*'s Console, rewrite Q and screen-shot it. Then embed the screen-shot in your document just below the query written in SQL (cf. previous point).
   NOTE: You can screen-shot it together with the result of its execution, see (d) below.

   (c) Execute the query over the database **UniversityDB**.

   (d) Screen-shot the result of execution (table' schema and instance). Two cases arise:

   Case 1. Execution leads to result. Then embed the screen-shot of such result in your document just below the screen-shot of the query you embedded (cf. point (b) above).

   Case 2. Execution leads to "syntax error" or any other execution error. Then:

   - debug your query (refer to MYSQL's manual, if necessary) and execute it again (and again, if necessary) until you managed to solve the error and correctly execute the query;

   - screen-shot **both the query you finally executed and its result** (table' schema and instance); embed it into your document just below the screen-shot of the query you embedded in (b) above (i.e., the original query Q);

   - explain in short what kind of issue you had to correctly execute the query. Be brief yet precise to identify the issue(s) you faced.

   NB: If, after several debugging cycles, you could not execute the query, explain the reasons you could not execute the query even if after careful debugging.

2. What is a database *view*? Reminder: A *view* is a table obtained by SQL standard statement:

   ```
   CREATE VIEW name_view AS (<query expression>);
   ```

   2.1 What are some advantages views have over base tables (i.e., tables stored in the database)?

3. What is a primary key? What characteristics does a good primary key have?

4. Describe at your best major similarities and differences between a `WHERE` clause and a `HAVING` clause in SQL standard.

5. Describe at your best what a foreign key is and how it relates to a primary key.

6. For each English query listed in "List of queries" below, do:

   (a) Write a *linguistically equivalent* SQL query on **UniversityDB** (cf. Figure 1, last page) by using SQL standard (i.e., SQL discussed in class and in the slides and videos).

   > **Definition:** Two queries on a database DB are *linguistically equivalent* if they represent (a) the same schema, (b) the same data instance, and (c) use identical language or semantically equivalent language.
   > **Example:** The SQL query `SELECT ID FROM student;` is *linguistically equivalent* to English query "Retrieve the ID from table about students".

   List of queries:

   i. By using an appropriate subquery in the appropriate SQL clause, find all sections that had the maximum enrollment and display each of then by section id and enrollment.
   ii. Find all students who have two or more F grades as per the $takes$ relation, and list them along with the F grades. Display the identifier (primary key) of each student.
   iii. Find all rooms that have been assigned to more than one section at the same time. Display the rooms in descending order of capacity along with the assigned sections.
   iv. Find the maximum and minimum enrollment across all sections, considering only sections that had some enrollment. (Don't worry about those that had no students taking that section.)
   v. Find all sections that had the maximum enrollment (along with the enrollment), using a subquery. Display the identifier (primary key) of each section.
   vi. Find all courses whose identifier (i.e., primary key) starts with the string "CS-". Display identifier and title of each course.
   vii. Find instructors who have taught all the above courses by using the "`NOT EXISTS ... EXCEPT ...`" combination of SQL statements.
   viii. Delete all data of students who got grade "F" in four (academic) years.
   ix. Insert all data of students you deleted in the previous query viii.
   x. Write a transaction to correctly update the total credits of a student (choose the student) who passed an exam such student took (cf. table `takes`).
   xi. Write a transaction to correctly update two accounts, each modelled by using a table, say `account1` and `account2` respectively, in case of a money transfer of amount 1000 USD from account with data in `account1` to account with data in `account2`.
   xii. Write a view that represents, in decreasing order by sum of credits, the ID, name and sum of credits computed department by department (of each student) that students obtained from taking and passing the courses "Database" and "Database Design" in Fall 2023 and Spring 2024, respectively.

7. For each SQL clause C listed in "List of SQL clauses" below[1], do:

   (a) By using C, write a SQL query (using SQL stadard) on the "University DB" (cf. Figure 1). Note: Be creative. Be interesting.

List of SQL clauses:

i. `LIKE` (Simple pattern matching, case sensitive)
ii. `iLIKE` (Simple pattern matching, case insensitive)
iii. `LIMIT` (constrain on number of rows returned; use with and without offset)
iv. `ROUND()` (numeric function; Round the argument)
v. `TRUNCATE()` (numeric function; Truncate to specified number of decimal places)
vi. `WITH` (cf. Common Table Expressions)
vii. `CONCAT()` (string function; Return concatenated string)
viii. `LENGTH()` (string function; Return the length of a string in bytes)
ix. `REPEAT()` (string function; Repeat a string the specified number of times)
x. `REPLACE()` (string function; Replace occurrences of a specified string)
xi. `REVERSE()` (string function; Reverse the characters in a string)
xii. `TIMEDIFF()` (data & time function; Subtract time)
xiii. `TIMESTAMP()` (data & time function)
xiv. `CURRENT_DATE()` (data & time function; Return the current date)
xv. `CURRENT_TIME()` (data & time function; Return the current time)

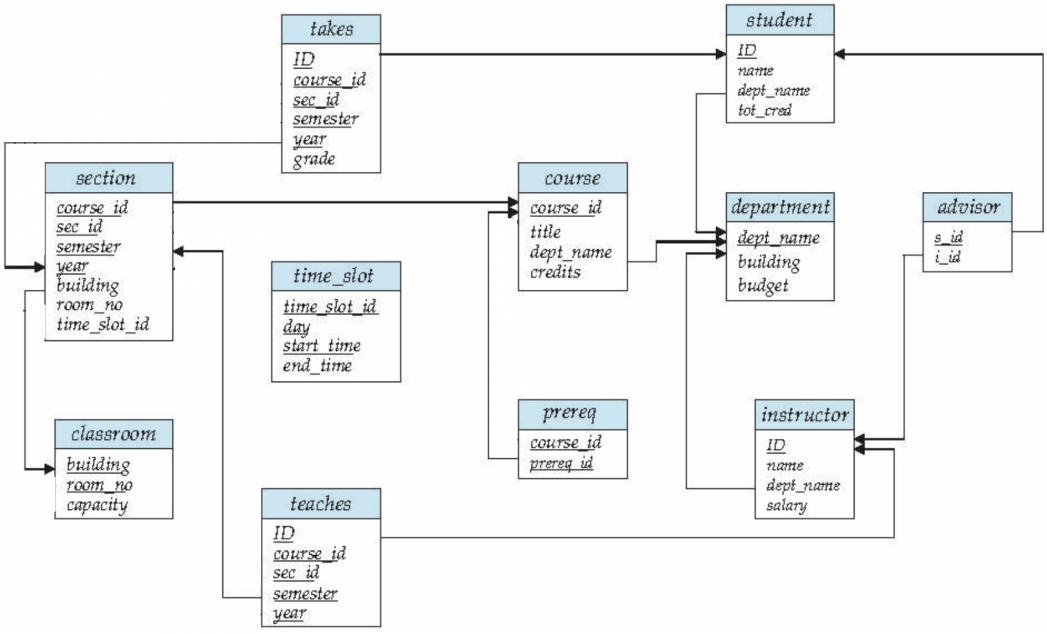

Figure 1: University Database (logical schema).

End of Homework 2

[1] In parentheses is the intended meaning of the clause as defined in MYSQL's manual.